

SQL 100 Questions - Data Engineer Prep

100 SQL Interview Questions for Data Engineers — Asked the Way Real Interviewers Ask Them

Sample Tables Used Throughout This Guide

- 📁 Round 1: SQL Basics & SELECT
- 📁 Round 2: Filtering, Sorting & CASE Logic
- 📁 Round 3: Aggregates, GROUP BY & HAVING
- 📁 Round 4: Joins
- 📁 Round 5: Subqueries
- 📁 Round 6: Window Functions
- 📁 Round 7: CTEs & Recursive Queries
- 📁 Round 8: String & Date Functions
- 📁 Round 9: Data Engineering Specific Scenarios
- 📁 Round 10: Real-World Business Scenarios & Performance
- 📖 How to Practice This Effectively

100 SQL Interview Questions for Data Engineers — Asked the Way Real Interviewers Ask Them

Format: 🗨️ **Interviewer asks** (exactly how it sounds in a live interview) → **Your Answer (query)** → **How to explain it out loud** (What/Why/How, so you sound confident, not memorized)

Sample Tables Used Throughout This Guide

```
-- employees(emp_id, name, dept_id, salary, manager_id, hire_date)
-- departments(dept_id, dept_name)
-- orders(order_id, customer_id, order_date, amount, status)
-- customers(customer_id, name, city, signup_date)
-- products(product_id, product_name, category, price)
-- order_items(order_id, product_id, quantity)
```

📁 Round 1: SQL Basics & SELECT

🗨️ **Q1. “We have an `employees` table. Can you write a query to pull everyone earning more than 50,000?”**

```
SELECT * FROM employees WHERE salary > 50000;
```

- **What it does:** Returns every column, for every employee, where `salary` is greater than 50000.
- **Why:** This is the most basic filter — almost every real query starts by narrowing down millions of rows to just the ones you care about.
- **How it works:** The engine reads each row, checks the `WHERE` condition, and keeps only rows where it's `TRUE`.

🔗 Q2. “How would you get a list of all the distinct departments we have, without repeats?”

```
SELECT DISTINCT dept_id FROM employees;
```

- **What it does:** Returns each unique `dept_id` only once, even if hundreds of employees share it.
- **Why:** Useful when you want “which departments exist,” not a row per employee.
- **How it works:** SQL computes the full result first, then removes exact duplicate rows.

🔗 Q3. “Can you show me employee names in uppercase for a formatted export?”

```
SELECT UPPER(name) AS emp_name FROM employees;
```

- **What it does:** Converts each name to uppercase in the output only.
- **Why:** Common for standardizing display or for case-insensitive comparisons later.
- **How it works:** `UPPER()` runs once per row and transforms just that row’s value — the actual stored data is untouched.

🔗 Q4. “Give me the top 5 highest-paid employees in the company.”

```
SELECT * FROM employees ORDER BY salary DESC LIMIT 5;
```

- **What it does:** Sorts everyone by salary descending, keeps only the first 5 rows.
- **Why:** “Top-N” queries are everywhere in reporting — top customers, top products, top earners.
- **How it works:** `ORDER BY` sorts first; `LIMIT` then cuts the sorted list down. (SQL Server: `SELECT TOP 5 ... ORDER BY salary DESC .`)

🔗 Q5. “Find me all employees whose name starts with the letter A.”

```
SELECT * FROM employees WHERE name LIKE 'A%';
```

- **What it does:** Matches any name beginning with “A,” regardless of what follows.
- **Why:** Pattern matching is needed whenever you know only PART of a value.
- **How it works:** `%` means “zero or more of any character.” `_` matches exactly one character instead.

🔗 Q6. “Pull everyone hired during the year 2023.”

```
SELECT * FROM employees
WHERE hire_date >= '2023-01-01' AND hire_date < '2024-01-01';
```

- **What it does:** Returns employees hired anywhere within that full year.
- **Why:** Date-range filtering is one of the most common real-world needs.
- **How it works:** Using `< '2024-01-01'` instead of `BETWEEN ... AND '2023-12-31'` avoids a classic gotcha — if `hire_date` has a time component, a hire at `2023-12-31 15:00` could get excluded by `BETWEEN`, since that’s technically past midnight of `'2023-12-31'`. The `< next-year` pattern is always safe.

🔗 Q7. “Some employees don’t have a manager assigned. How do you find them?”

```
SELECT * FROM employees WHERE manager_id IS NULL;
```

- **What it does:** Finds rows where `manager_id` is truly unknown/missing (e.g., the CEO).
- **Why:** NULL represents “unknown” — you can’t use `=` to check for it.
- **How it works:** NULL isn’t equal to anything, not even itself, so `manager_id = NULL` always returns nothing. You must use `IS NULL / IS NOT NULL`.

🔗 Q8. “In the report, can the `salary` column be labeled `monthly_salary` instead?”

```
SELECT name, salary AS monthly_salary FROM employees;
```

- **What it does:** Displays `salary` under a new label in the result only.
- **Why:** Makes reports readable for business users unfamiliar with raw column names.
- **How it works:** `AS` creates an alias — a display name only, the table column is untouched.

🔗 Q9. “The name is split into `first_name` and `last_name` — can you combine them into one ‘Full Name’ field?”

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM employees;
```

- **What it does:** Joins first name, a space, and last name into one string.
- **Why:** Source systems store names split; reports usually want them combined.
- **How it works:** `CONCAT()` glues arguments together left to right. (Postgres also supports `first_name || ' ' || last_name`.)

🔗 Q10. “Show me everyone NOT in department 10 or 20.”

```
SELECT * FROM employees WHERE dept_id NOT IN (10, 20);
```

- **What it does:** Returns employees whose department is anything other than 10 or 20.
- **Why:** Exclusion filters come up constantly — “everything except X.”
- **How it works:** ⚠️ Gotcha: if that list ever contains a NULL (e.g., from a subquery), the WHOLE query silently returns zero rows, because SQL can’t be certain about “not equal to NULL.” Watch for this when the list comes from another query.

📁 Round 2: Filtering, Sorting & CASE Logic

🔗 Q11. “Business wants employees bucketed into High/Medium/Low earners for a dashboard — how would you do that in SQL?”

```
SELECT name,
CASE
  WHEN salary > 80000 THEN 'High'
  WHEN salary BETWEEN 40000 AND 80000 THEN 'Medium'
  ELSE 'Low'
END AS salary_band
FROM employees;
```

- **What it does:** Creates a new `salary_band` column labeling each employee by range.
- **Why:** Business users think in categories, not raw numbers — `CASE` translates numbers into labels.
- **How it works:** SQL checks each `WHEN` top to bottom, per row; the first `TRUE` wins and the rest are skipped. `ELSE` catches anything unmatched.

🔗 Q12. “I want orders sorted by amount, highest first — and if two orders tie, older one first.”

```
SELECT * FROM orders ORDER BY amount DESC, order_date ASC;
```

- **What it does:** Sorts by amount descending; ties are broken by `order_date` ascending.
- **Why:** Real data has ties — without a tie-breaker, tied rows’ order is undefined and inconsistent across runs.
- **How it works:** Multiple `ORDER BY` columns act like nested sort levels — first sorts by `amount`, second only decides order WITHIN ties.

🔗 Q13. “Can you get me all orders from the last 30 days?”

```
SELECT * FROM orders WHERE order_date >= CURRENT_DATE - INTERVAL
'30 days';
```

- **What it does:** Filters to orders within the last month, relative to today.
- **Why:** “Rolling window” queries power almost every dashboard, auto-updating every run.
- **How it works:** `CURRENT_DATE` gets today; subtracting the interval gives the cutoff. (Syntax differs by engine — SQL Server: `DATEADD(day, -30, GETDATE())`.)

🔗 Q14. “Filter customers who are based in Hyderabad or Chennai.”

```
SELECT * FROM customers WHERE city IN ('Hyderabad', 'Chennai');
```

- **What it does:** Returns customers matching ANY listed city.
- **Why:** `IN` is a cleaner shortcut than chaining several `OR` s.
- **How it works:** Functionally identical to `city = 'Hyderabad' OR city = 'Chennai'`.

🔗 Q15. “Before we trust this orders table, can you check for anything that looks broken — like missing or zero amounts?”

```
SELECT * FROM orders WHERE amount IS NULL OR amount = 0;
```

- **What it does:** Flags orders with missing or zero value.
- **Why:** This is a data-quality check — the first thing a Data Engineer runs on a new source before writing transformation logic.
- **How it works:** Two conditions combined with `OR` — a row qualifies if EITHER is true.

⚡ Q16. “Get me mid-range products, priced 100 to 500, but skip anything in Electronics.”

```
SELECT * FROM products
WHERE price BETWEEN 100 AND 500 AND category <> 'Electronics';
```

- **What it does:** Both conditions must be true: in price range AND not Electronics.
- **Why:** Real filters rarely have just one condition.
- **How it works:** `AND` requires every condition to be `TRUE` for a row to qualify.

⚡ Q17. “Without using `LIMIT` or `OFFSET`, how would you find the 3rd highest salary?”

```
SELECT MIN(salary) FROM (
    SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT
3
) t;
```

- **What it does:** Gets the top 3 distinct salaries, then returns the smallest of those 3 — which IS the 3rd highest.
- **Why:** Classic trick question testing your understanding of subqueries.
- **How it works:** Inner query gets `[95000, 90000, 85000]`; outer `MIN()` on that mini-result returns `85000`.

⚡ Q18. “I need employees grouped visually by department, and within each department, highest earners on top.”

```
SELECT * FROM employees ORDER BY dept_id, salary DESC;
```

- **What it does:** Groups same-department rows together; within each block, sorts by salary descending.
- **Why:** Prepares data for a department-wise report using pure sorting, no aggregation.
- **How it works:** Sorts primarily by `dept_id`; `salary DESC` only orders rows within the same department.

⚡ Q19. “Search for any customer whose name has ‘raj’ in it anywhere — case shouldn’t matter.”

```
SELECT * FROM customers WHERE LOWER(name) LIKE '%raj%';
```

- **What it does:** Matches “raj” anywhere in the name, ignoring capitalization.
- **Why:** Real text data is inconsistent — “Raj,” “RAJ,” “raj Kumar” should all match.
- **How it works:** `LOWER(name)` normalizes case before comparing to the lowercase pattern `'%raj%'`.

⚡ Q20. “Show me all orders except the cancelled ones.”

```
SELECT * FROM orders WHERE status != 'CANCELLED';
```

- **What it does:** Excludes rows explicitly marked cancelled.
 - **Why:** Simple exclusion — “everything except one bad state.”
 - **How it works:** ⚠ Gotcha: rows where `status IS NULL` get silently dropped too, since `NULL != 'CANCELLED'` isn’t `TRUE` — it’s unknown. Add `OR status IS NULL` if you want to keep them.
-

📁 Round 3: Aggregates, GROUP BY & HAVING

🔗 Q21. “How much has each customer spent with us in total?”

```
SELECT customer_id, SUM(amount) AS total_revenue
FROM orders GROUP BY customer_id;
```

- **What it does:** Collapses all of a customer’s orders into one row showing their combined spend.
- **Why:** Foundation of almost every business report — “how much did each X do.”
- **How it works:** Rows sharing the same `customer_id` get bucketed together; `SUM(amount)` runs per bucket.

🔗 Q22. “Which departments have more than 10 employees?”

```
SELECT dept_id, COUNT(*) AS emp_count
FROM employees GROUP BY dept_id
HAVING COUNT(*) > 10;
```

- **What it does:** Counts employees per department, keeps only those over 10.
- **Why:** Tests the single most-confused SQL concept: filtering on an aggregate result.
- **How it works:** `WHERE` filters rows BEFORE grouping (none here); `GROUP BY` buckets by department; `COUNT(*)` runs per bucket; `HAVING` filters groups AFTER aggregation. You can’t write `WHERE COUNT(*) > 10` — it doesn’t exist yet at that stage.

🔗 Q23. “What’s the average order value, broken down by month?”

```
SELECT DATE_TRUNC('month', order_date) AS month, AVG(amount) AS
avg_order_value
FROM orders GROUP BY DATE_TRUNC('month', order_date);
```

- **What it does:** Buckets orders into calendar months, averages each month’s amounts.
- **Why:** Trend analysis needs data rolled up to a coarser time grain.
- **How it works:** `DATE_TRUNC('month', order_date)` strips day/time, leaving the first of the month, so same-month orders group together.

🔗 Q24. “For each product, how many DIFFERENT customers actually bought it — not order count, unique buyers?”

```
SELECT product_id, COUNT(DISTINCT customer_id) AS unique_buyers
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
GROUP BY product_id;
```

- **What it does:** Counts distinct customers per product, not raw order rows.
- **Why:** One loyal repeat buyer shouldn’t inflate the “unique buyers” number.
- **How it works:** `COUNT(DISTINCT ...)` removes duplicate `customer_ids` within each group before counting.

🔗 Q25. “Show me each customer’s biggest and smallest order side by side.”

```
SELECT customer_id, MAX(amount) AS max_order, MIN(amount) AS
min_order
FROM orders GROUP BY customer_id;
```

- **What it does:** Per customer, shows max and min order amount together.
- **Why:** Multiple aggregates can be computed in a single pass — no need for separate queries.
- **How it works:** `MAX()` and `MIN()` both operate on the same grouped bucket simultaneously.

🔗 Q26. “Which departments have a total payroll over 5 lakh?”

```
SELECT dept_id, SUM(salary) AS total_salary
FROM employees GROUP BY dept_id
HAVING SUM(salary) > 500000;
```

- **What it does:** Sums salaries per department, keeps only those over ₹5,00,000.
- **Why:** Same `HAVING` concept as Q22, now on `SUM` instead of `COUNT`.
- **How it works:** You can put the FULL aggregate expression directly inside `HAVING`.

🔗 Q27. “Give me a breakdown of order counts by status.”

```
SELECT status, COUNT(*) AS order_count
FROM orders GROUP BY status;
```

- **What it does:** Counts orders per status category.
- **Why:** The simplest “distribution” report — instantly shows operational health.
- **How it works:** Every unique `status` value becomes its own group.

🔗 Q28. “Find customers who ordered more than 5 times in the last 90 days — for a loyalty campaign.”

```
SELECT customer_id, COUNT(*) AS order_count
FROM orders
WHERE order_date >= CURRENT_DATE - INTERVAL '90 days'
GROUP BY customer_id
HAVING COUNT(*) > 5;
```

- **What it does:** Filters to recent orders first, then counts per customer, then keeps frequent buyers.
- **Why:** Real questions are usually multi-layered — row filter + grouping + group filter, all together.
- **How it works (execution order):** `WHERE` runs first (drops old orders) → `GROUP BY` buckets what remains → `COUNT(*)` computes per bucket → `HAVING` filters the buckets last.

🔗 Q29. “For the leadership deck, show each category’s revenue AND what percent of total revenue it represents.”

```
SELECT category,
SUM(price * quantity) AS category_revenue,
SUM(price * quantity) * 100.0 / SUM(SUM(price * quantity)) OVER
() AS pct_of_total
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY category;
```

- **What it does:** Shows each category’s revenue and its share of the

grand total.

- **Why:** Executives want percentages, not just raw totals.
- **How it works:** `SUM(price*quantity)` computes each category's own total; `SUM(SUM(...)) OVER ()` (empty parens = no partition) adds up ALL category totals into one grand total, repeated on every row, so dividing gives the share.

🔗 Q30. "Who is our SECOND highest-spending customer, handling any ties correctly?"

```
SELECT customer_id, total_revenue FROM (  
  SELECT customer_id, SUM(amount) AS total_revenue,  
         DENSE_RANK() OVER (ORDER BY SUM(amount) DESC) AS rn  
  FROM orders GROUP BY customer_id  
 ) t WHERE rn = 2;
```

- **What it does:** Ranks customers by total spend, picks whoever lands at rank 2.
- **Why:** Shows why `DENSE_RANK()` beats plain `LIMIT/OFFSET` when ties are possible.
- **How it works:** If two customers tie for 1st, BOTH get rank 1, and the next distinct spender correctly gets rank 2 — no rank is skipped or duplicated incorrectly.

📁 Round 4: Joins

🔗 Q31. "I need each order with the customer's name attached, not just their ID."

```
SELECT o.order_id, c.name, o.amount  
FROM orders o  
INNER JOIN customers c ON o.customer_id = c.customer_id;
```

- **What it does:** Combines orders with customers, matching on `customer_id`.
- **Why:** Data is split across tables for storage efficiency; reports need it flattened back.
- **How it works:** For every order, SQL finds the matching customer row. If NO match exists, that order is dropped — the defining trait of `INNER JOIN`.

🔗 Q32. "Show me ALL customers, even the ones who've never ordered anything."

```
SELECT c.customer_id, c.name, o.order_id  
FROM customers c  
LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

- **What it does:** Keeps every customer; unmatched order columns show as NULL.
- **Why:** Fixes `INNER JOIN`'s "drops unmatched rows" behavior when you specifically want to see the gaps.
- **How it works:** `LEFT JOIN` keeps ALL rows from the left table no matter what, filling unmatched right-side columns with NULL instead of dropping the row.

🔗 Q33. "Which customers have NEVER placed an order? I want a list for the marketing team."

```
SELECT c.customer_id, c.name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

- **What it does:** Keeps only the customers where the order-side came back empty.
- **Why:** “LEFT JOIN + IS NULL” is THE standard pattern for finding “things in A with no relationship in B” (an anti-join).
- **How it works:** Matched customers get a real `order_id`; unmatched ones get NULL — filtering on that NULL isolates the never-ordered group.

🔗 Q34. “List every department along with its employees — but I still want to see departments that currently have zero staff.”

```
SELECT d.dept_name, e.name
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id;
```

- **What it does:** Keeps every department; unmatched employee columns show NULL.
- **Why:** Direction matters — whichever table comes right after `FROM` is the “keep everything” side.
- **How it works:** Here `departments` is on the left (kept fully), `employees` fills in only where matched.

🔗 Q35. “For the report grid, I need every product paired with every month, even combos with zero sales.”

```
SELECT p.product_id, m.month
FROM products p
CROSS JOIN (SELECT DISTINCT DATE_TRUNC('month', order_date) AS
month FROM orders) m;
```

- **What it does:** Pairs every product with every month with no matching condition.
- **Why:** Report tools need a complete grid so zero-activity combos still show as 0, not missing.
- **How it works:** `CROSS JOIN` produces every possible pairing — 50 products × 12 months = 600 rows — typically then LEFT JOINED to real sales data.

🔗 Q36. “Can you show me each employee next to their manager’s actual name, not just the ID?”

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

- **What it does:** Looks up a manager’s name by matching `manager_id` to that manager’s own `emp_id`, in the same table.
- **Why:** Hierarchical data (org charts) is often self-referencing within one table.
- **How it works:** Using two aliases (`e`, `m`) for the same table lets SQL treat it as if it were two tables. `LEFT JOIN` keeps top-level people (like a CEO with no manager) with `manager = NULL`.

🔗 Q37. “Are there any orders in our system referencing a customer that doesn’t actually exist? I want a data quality check.”

```
SELECT o.*
FROM orders o
LEFT JOIN customers c ON o.customer_id = c.customer_id
WHERE c.customer_id IS NULL;
```

- **What it does:** Same anti-join pattern as Q33, flipped to catch broken orders.
- **Why:** This is a referential integrity check — critical before loading data into a trusted Gold layer.
- **How it works:** Unmatched orders show `c.customer_id = NULL`, exposing orphaned references.

🔗 **Q38. “I want to reconcile employees and departments both ways — mismatches on either side.”**

```
SELECT e.name, d.dept_name
FROM employees e
FULL OUTER JOIN departments d ON e.dept_id = d.dept_id;
```

- **What it does:** Shows every employee AND every department, even unmatched ones from both sides.
- **Why:** Catches orphaned records in both directions in one query, instead of two separate LEFT JOINs.
- **How it works:** `FULL OUTER JOIN` = `LEFT JOIN` + `RIGHT JOIN` combined. (MySQL doesn’t support it natively — simulate with `LEFT JOIN UNION RIGHT JOIN`.)

🔗 **Q39. “Calculate the total value of each order — quantity times price, summed up.”**

```
SELECT oi.order_id, SUM(oi.quantity * p.price) AS order_total
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY oi.order_id;
```

- **What it does:** Attaches price from `products`, multiplies by quantity, sums per order.
- **Why:** Classic “enrich then aggregate” pattern — `order_items` alone doesn’t have price.
- **How it works:** `JOIN` attaches price → line-item subtotal calculated → `GROUP BY` + `SUM` totals per order.

🔗 **Q40. “Build me one flat table with order, customer name, and product details all together — for Power BI.”**

```
SELECT o.order_id, c.name, oi.product_id, oi.quantity
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id = oi.order_id;
```

- **What it does:** Chains multiple joins to pull customer + item details into one wide table.
- **Why:** BI tools want a flat, denormalized table rather than requiring the report-builder to join tables themselves.
- **How it works:** Each JOIN processes in sequence; the engine figures out the most efficient execution order internally. One order with 3 products naturally produces 3 rows (expected “fan-out”).

📁 Round 5: Subqueries

⚡ Q41. “Show me employees earning above the company average salary.”

```
SELECT * FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

- **What it does:** Compares each salary to the one company-wide average.
- **Why:** You need to compare rows to a calculated benchmark, not a hardcoded number.
- **How it works:** The inner query runs once, produces ONE number (a “scalar subquery”), and that number gets substituted into the outer comparison.

⚡ Q42. “Which customers have placed at least one order above ₹5000?”

```
SELECT * FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders WHERE amount
> 5000);
```

- **What it does:** Builds a list of qualifying customer_ids from `orders`, then filters `customers` against it.
- **Why:** Lets you filter one table based on a condition living in a related table.
- **How it works:** Inner query returns a LIST; outer `IN` checks membership against that list.

⚡ Q43. “Find products that have never been ordered — any dead stock?”

```
SELECT * FROM products
WHERE product_id NOT IN (SELECT product_id FROM order_items);
```

- **What it does:** Finds products absent from `order_items`.
- **Why:** Spots dead stock — products that never sold.
- **How it works:** ⚠ Big gotcha: if even ONE `order_items.product_id` is NULL, this ENTIRE query silently returns zero rows, because SQL can’t prove “not equal to NULL” for every comparison.

⚡ Q44. “Same question — but I’ve heard NOT IN can be risky with NULLs. How would you write it safely?”

```
SELECT * FROM products p
WHERE NOT EXISTS (
    SELECT 1 FROM order_items oi WHERE oi.product_id = p.product_id
);
```

- **What it does:** Same goal as Q43, avoiding the NULL trap.
- **Why:** The professional, production-safe way to write “not in” logic.
- **How it works:** For each product, checks “does at least one matching row exist?” — `EXISTS` only cares about TRUE/FALSE, so NULLs never cause the same problem.

⚡ Q45. “Find each department’s highest-paid employee — using a subquery, not a window function.”

```
SELECT * FROM employees e
WHERE salary = (
    SELECT MAX(salary) FROM employees e2 WHERE e2.dept_id =
e.dept_id
);
```

- **What it does:** For each employee, checks if their salary equals the max within their own department.
- **Why:** Introduces correlated subqueries — where the inner query depends on the outer row.
- **How it works:** This inner query re-runs ONCE PER outer row, recalculating a department-specific max each time, because `e2.dept_id = e.dept_id` ties it to whichever row is currently being checked. (Note: can be slow at scale — see Q52 for a faster window-function version.)

🔗 Q46. “Get me the customers whose total spend is over ₹1,00,000 — but write it using a subquery in the FROM clause.”

```
SELECT * FROM (
    SELECT customer_id, SUM(amount) AS total_spent
    FROM orders GROUP BY customer_id
) t
WHERE total_spent > 100000;
```

- **What it does:** Same result as using HAVING, via an alternate technique.
- **Why:** Sometimes you need to do further processing on top of an already-aggregated result, which a derived table makes easy.
- **How it works:** The inner aggregation is treated as a virtual table (`t`); the outer `WHERE` filters it like any normal table.

🔗 Q47. “Find employees who earn more than their OWN department’s average — not the company average.”

```
SELECT * FROM employees e
WHERE salary > (
    SELECT AVG(salary) FROM employees e2 WHERE e2.dept_id =
e.dept_id
);
```

- **What it does:** Compares each employee to their own department’s average.
- **Why:** Shows correlated subqueries comparing a row to a group-specific benchmark.
- **How it works:** Inner query recalculates per outer row, scoped to that row’s `dept_id` via the correlation condition.

🔗 Q48. “Before inserting a new customer, how do you check if they already exist?”

```
SELECT 1 WHERE EXISTS (
    SELECT 1 FROM customers WHERE customer_id = 101
);
```

- **What it does:** Checks existence without needing the actual data.
- **Why:** ETL code needs fast existence checks to avoid creating duplicates.
- **How it works:** `EXISTS` stops scanning the moment ONE match is found — fast, especially with an index on `customer_id`.

🔗 Q49. “Get me each customer’s most recent order — using a subquery approach.”

```

SELECT * FROM orders o
WHERE order_date = (
    SELECT MAX(order_date) FROM orders o2 WHERE o2.customer_id =
o.customer_id
);

```

- **What it does:** Keeps rows where the order date matches that customer's own latest date.
- **Why:** "Latest record per group" is an extremely common real-world need.
- **How it works:** Correlated subquery recalculates the max date per customer. ⚠ Note: if a customer has a tie (two orders on the exact same latest date), BOTH rows survive here — unlike the `ROW_NUMBER()` version in Q56, which would guarantee exactly one.

🔗 Q50. "Which departments beat the overall company average salary?"

```

SELECT dept_id, AVG(salary) AS dept_avg
FROM employees
GROUP BY dept_id
HAVING AVG(salary) > (SELECT AVG(salary) FROM employees);

```

- **What it does:** Compares each department's average against the one overall company average.
- **Why:** Combines GROUP BY/HAVING with a scalar subquery — exactly how layered real questions get built.
- **How it works:** The subquery here is NOT correlated — it calculates ONE fixed company-wide number, used as the benchmark for every department group.

📁 Round 6: Window Functions

This is usually where interviewers spend the MOST time — take it slow and really understand the "why," not just the syntax.

🔗 Q51. "I need every employee visible in the result, but with a column showing their salary rank within their own department."

```

SELECT name, dept_id, salary,
    RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS
salary_rank
FROM employees;

```

- **What it does:** Every row stays visible, with an added rank-within-department column.
- **Why:** You often need "rank within a group" without collapsing rows — a plain GROUP BY would force you to lose row-level detail.
- **How it works:** `PARTITION BY dept_id` resets the calculation per department (like an invisible GROUP BY that doesn't collapse rows); `ORDER BY salary DESC` sorts within each partition; `RANK()` assigns 1, 2, 3... but ties share the SAME rank and the next rank number gets skipped (1, 1, 3).

🔗 Q52. "Same 'top earner per department' question as before — but can you do it more efficiently, without a correlated subquery?"

```

SELECT * FROM (
    SELECT *, RANK() OVER (PARTITION BY dept_id ORDER BY salary
DESC) AS rnk
    FROM employees
) t WHERE rnk = 1;

```

- **What it does:** Same goal as Q45, computed via a window function instead.
- **Why:** Window functions run in a SINGLE pass over the data; correlated subqueries re-scan once PER row — a big performance difference at scale.
- **How it works:** Ranks are assigned per department (like Q51), then filtered down to `rnk = 1`. Ties at the top both correctly appear.

🔗 Q53. “I want to see the running total of revenue as each day progresses.”

```

SELECT order_date, SUM(amount) AS daily_total,
    SUM(SUM(amount)) OVER (ORDER BY order_date) AS running_total
FROM orders
GROUP BY order_date;

```

- **What it does:** Shows each day’s total AND a cumulative total that keeps growing.
- **Why:** Running totals are core to finance reporting — “how much have we earned so far.”
- **How it works:** `SUM() OVER (ORDER BY order_date)` with NO `PARTITION BY` treats the whole result as one group; adding an `ORDER BY` inside `OVER()` triggers the default cumulative behavior — each row sums everything from the start up to itself.

🔗 Q54. “For each customer, show how their latest order compares to the one before it.”

```

SELECT customer_id, order_date, amount,
    amount - LAG(amount) OVER (PARTITION BY customer_id ORDER BY
order_date) AS diff_from_prev
FROM orders;

```

- **What it does:** Looks backward to the customer’s previous order and calculates the difference.
- **Why:** Trend/change analysis requires comparing a row to a neighboring row — impossible with normal aggregates.
- **How it works:** `LAG(amount)` fetches the value from one row before, within that customer’s own ordered partition. The very first order of each customer gets NULL (nothing before it).

🔗 Q55. “Can you tell me the NEXT order date for each customer, based on their current one?”

```

SELECT customer_id, order_date,
    LEAD(order_date) OVER (PARTITION BY customer_id ORDER BY
order_date) AS next_order_date
FROM orders;

```

- **What it does:** The mirror of Q54 — looks forward instead of backward.
- **Why:** Useful for “days until next purchase” or gap analysis.
- **How it works:** `LEAD()` fetches the value from the row that comes AFTER, within the same partition/order. The last order per customer returns NULL.

⚡ Q56. “Our source system has multiple versions of the same customer record. How do you keep only the LATEST one per customer?”

```
SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY
order_date DESC) AS rn
    FROM orders
) t WHERE rn = 1;
```

- **What it does:** Numbers each customer's rows starting from the most recent, keeps only rank 1.
- **Why:** THE standard deduplication pattern in real data pipelines.
- **How it works:** `PARTITION BY customer_id` restarts numbering per customer; `ORDER BY order_date DESC` sorts newest-first; `ROW_NUMBER()` gives strictly unique numbers (no ties allowed, unlike RANK) — guaranteeing exactly one row per customer even with an exact date tie.

⚡ Q57. “For a smoother trend line, can you compute a 3-day moving average of order amounts?”

```
SELECT order_date, amount,
    AVG(amount) OVER (ORDER BY order_date ROWS BETWEEN 2 PRECEDING
AND CURRENT ROW) AS moving_avg_3d
FROM orders;
```

- **What it does:** Averages each row with the 2 rows before it — a sliding 3-row window.
- **Why:** Moving averages smooth out day-to-day noise, revealing the real trend.
- **How it works:** `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` explicitly defines the frame; it slides forward, recalculated fresh for every row.

⚡ Q58. “Marketing wants customers split into 4 spending tiers — top 25%, next 25%, and so on.”

```
SELECT customer_id, SUM(amount) AS total_spent,
    NTILE(4) OVER (ORDER BY SUM(amount) DESC) AS quartile
FROM orders GROUP BY customer_id;
```

- **What it does:** Sorts customers by spend, splits them into 4 roughly equal buckets.
- **Why:** Customer segmentation for marketing strategy almost always uses quartiles like this.
- **How it works:** `NTILE(4)` divides the ordered row count by 4, assigning bucket numbers as evenly as possible.

⚡ Q59. “For each order, show what percentage it represents of that customer's total spend — without collapsing to one row per customer.”

```
SELECT customer_id, order_id, amount,
    amount * 100.0 / SUM(amount) OVER (PARTITION BY customer_id) AS
pct_of_customer_total
FROM orders;
```

- **What it does:** Shows each order's share of that customer's overall spend, row by row.
- **Why:** A perfect example of needing BOTH a group total AND individual row detail together — a plain GROUP BY can't give you

both.

- **How it works:** `SUM(amount) OVER (PARTITION BY customer_id)` with `NO ORDER BY` computes the FULL partition total (not cumulative), repeating that same total on every row for that customer.

🔗 Q60. “Alongside every order, I want to see that customer’s very first order date, and their most recent one.”

```
SELECT *,
    FIRST_VALUE(order_date) OVER (PARTITION BY customer_id ORDER BY
order_date) AS first_order,
    LAST_VALUE(order_date) OVER (PARTITION BY customer_id ORDER BY
order_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS
last_order
FROM orders;
```

- **What it does:** Adds first-order and last-order dates to every row.
- **Why:** Useful for customer lifetime calculations without a separate query.
- **How it works:** `FIRST_VALUE` naturally works since the partition is ordered ascending. ⚠ Classic gotcha: `LAST_VALUE` needs an EXPLICIT full-frame (`ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`) — without it, the default frame only looks up to the current row, so `LAST_VALUE` would confusingly just return the current row’s own date every time.

📁 Round 7: CTEs & Recursive Queries

🔗 Q61. “This query is getting nested and hard to read — can you clean it up using a CTE?”

```
WITH customer_revenue AS (
    SELECT customer_id, SUM(amount) AS total_revenue
    FROM orders GROUP BY customer_id
)
SELECT * FROM customer_revenue WHERE total_revenue > 50000;
```

- **What it does:** Same result as Q46’s subquery, but written top-to-bottom with `WITH`.
- **Why:** CTEs are far easier to read than nested subqueries, especially as logic grows.
- **How it works:** `WITH customer_revenue AS (...)` names a temporary result you reference like a real table in the main query.

🔗 Q62. “Can you break this multi-step logic into clear stages instead of one giant query?”

```
WITH step1 AS (
    SELECT customer_id, SUM(amount) AS total_spent FROM orders GROUP
BY customer_id
),
step2 AS (
    SELECT customer_id, total_spent,
        CASE WHEN total_spent > 100000 THEN 'VIP' ELSE 'Regular' END
AS segment
    FROM step1
)
SELECT * FROM step2 WHERE segment = 'VIP';
```

- **What it does:** Calculates totals (step1), labels them (step2), filters

(final SELECT) — in clear stages.

- **Why:** Mirrors real pipeline structure (Bronze → Silver → Gold), compressed into one query.
- **How it works:** Each CTE can reference the ones defined before it — `step2` uses `step1`, not the reverse.

🔗 **Q63. “We have an employee-manager hierarchy of unknown depth. How would you build a full org chart with levels?”**

```
WITH RECURSIVE org_chart AS (  
    SELECT emp_id, name, manager_id, 1 AS level  
    FROM employees WHERE manager_id IS NULL  
    UNION ALL  
    SELECT e.emp_id, e.name, e.manager_id, oc.level + 1  
    FROM employees e  
    JOIN org_chart oc ON e.manager_id = oc.emp_id  
)  
SELECT * FROM org_chart;
```

- **What it does:** Traverses the hierarchy top-down, assigning a level to each employee.
- **Why:** Tree-shaped data (org charts, category trees, BOMs) needs recursion — a normal query can't traverse unknown depth.
- **How it works:** The anchor finds the top (`manager_id IS NULL`, level 1). The recursive part joins back to the CTE itself, finding the next level down each round, repeating until a round finds zero new matches.

🔗 **Q64. “For each city, give me the top 3 highest-spending customers.”**

```
WITH ranked AS (  
    SELECT c.city, c.customer_id, SUM(o.amount) AS total_spent,  
           RANK() OVER (PARTITION BY c.city ORDER BY SUM(o.amount) DESC)  
AS rnk  
    FROM customers c JOIN orders o ON c.customer_id = o.customer_id  
    GROUP BY c.city, c.customer_id  
)  
SELECT * FROM ranked WHERE rnk <= 3;
```

- **What it does:** Finds the top 3 spenders per city.
- **Why:** Combines JOIN, GROUP BY, SUM, RANK, and a CTE — exactly what real work questions look like.
- **How it works:** The CTE computes rank within each city; you CAN'T filter `WHERE rnk <= 3` in the same query where `rnk` is calculated, so wrapping it in a CTE gives you a layer where `rnk` exists as a normal column to filter on.

🔗 **Q65. “We loaded duplicate order rows by mistake — how would you clean them up, keeping the earliest copy?”**

```
WITH dedup AS (  
    SELECT *, ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY  
order_date) AS rn  
    FROM orders  
)  
DELETE FROM orders WHERE order_id IN (  
    SELECT order_id FROM dedup WHERE rn > 1  
);
```

- **What it does:** Identifies duplicate order_ids, deletes all but the earliest.

- **Why:** Sometimes duplicates need permanent removal, not just filtering in a SELECT.
- **How it works:** Same `ROW_NUMBER()` logic as Q56 — `rn = 1` is kept, `rn > 1` gets deleted. (Exact CTE+DELETE syntax varies by engine.)

🔗 Q66. “Give me month-over-month revenue growth percentage.”

```
WITH monthly AS (
  SELECT DATE_TRUNC('month', order_date) AS month, SUM(amount) AS
revenue
  FROM orders GROUP BY DATE_TRUNC('month', order_date)
)
SELECT month, revenue,
  (revenue - LAG(revenue) OVER (ORDER BY month)) * 100.0 /
LAG(revenue) OVER (ORDER BY month) AS growth_pct
FROM monthly;
```

- **What it does:** Aggregates monthly revenue, then compares each month to the previous one.
- **Why:** One of the most commonly requested KPIs in any business dashboard.
- **How it works:** CTE handles the monthly aggregation; `LAG(revenue)` grabs the prior month for the growth formula.

🔗 Q67. “Flag any customer who hasn’t ordered in the last 60 days — churn risk list.”

```
WITH last_order AS (
  SELECT customer_id, MAX(order_date) AS last_order_date
  FROM orders GROUP BY customer_id
)
SELECT * FROM last_order
WHERE last_order_date < CURRENT_DATE - INTERVAL '60 days';
```

- **What it does:** Finds each customer’s last order date, flags those over 60 days old.
- **Why:** Foundation of real churn-risk alerts used by retention teams.
- **How it works:** Simple `MAX()` per customer in the CTE, filtered against today minus 60 days.

🔗 Q68. “For the calendar report, I need every date in January — even days with zero orders shouldn’t disappear.”

```
WITH RECURSIVE date_series AS (
  SELECT DATE '2026-01-01' AS dt
  UNION ALL
  SELECT dt + INTERVAL '1 day' FROM date_series WHERE dt < DATE
'2026-01-31'
)
SELECT * FROM date_series;
```

- **What it does:** Generates every calendar date from Jan 1 to Jan 31, even though no dates table exists.
- **Why:** Grouping directly on `order_date` makes zero-order days disappear, breaking charts with visible gaps.
- **How it works:** Anchor starts at Jan 1; recursive part adds a day each round until the boundary condition fails. Typically LEFT JOINed to real order data afterward so missing days show 0.

🔗 Q69. “Which products make up 80% of our total revenue — an 80/20 analysis?”

```

WITH product_rev AS (
    SELECT product_id, SUM(quantity * price) AS revenue
    FROM order_items oi JOIN products p ON oi.product_id =
p.product_id
    GROUP BY product_id
),
ranked AS (
    SELECT *, SUM(revenue) OVER (ORDER BY revenue DESC) * 1.0 /
SUM(revenue) OVER () AS cumulative_pct
    FROM product_rev
)
SELECT * FROM ranked WHERE cumulative_pct <= 0.8;

```

- **What it does:** Sorts products by revenue, tracks a running percentage, stops at 80%.
- **Why:** The classic Pareto principle — knowing which products deserve the most attention.
- **How it works:** A running total (ordered by revenue) divided by the grand total (unordered, same on every row) gives the cumulative percentage as you move down the list.

🔗 **Q70. “Our pipeline reloads the whole table every day and it’s too slow. How would you make it incremental?”**

```

WITH source_data AS (
    SELECT * FROM staging_orders WHERE updated_at > (SELECT
MAX(last_load_ts) FROM etl_control)
)
SELECT * FROM source_data;

```

- **What it does:** Only pulls rows updated after the last successful pipeline run.
- **Why:** Reloading massive tables daily is slow and wasteful — incremental loading only processes what changed.
- **How it works:** A small control table stores the last “watermark” timestamp; the filter only keeps rows newer than it. After a successful run, the control table gets updated with a new watermark (see Q86) for the next run.

📁 Round 8: String & Date Functions

🔗 **Q71. “Can you break out the year and month separately from the order_date column?”**

```

SELECT order_id, EXTRACT(YEAR FROM order_date) AS yr,
EXTRACT(MONTH FROM order_date) AS mth
FROM orders;

```

- **What it does:** Pulls year and month as separate numbers from a date value.
- **Why:** Reports often need year/month filtering or grouping separately from the exact date.
- **How it works:** `EXTRACT(part FROM date)` is standard ANSI SQL for pulling a specific date component.

🔗 **Q72. “What’s the average delivery time — days between order date and delivery date?”**

```

SELECT order_id, delivery_date - order_date AS days_to_deliver
FROM orders;

```

- **What it does:** Subtracts one date from another to get elapsed days.
- **Why:** Measuring durations is a common operational metric.
- **How it works:** In Postgres, subtracting two DATE columns gives a plain integer directly. (SQL Server needs `DATEDIFF(day, order_date, delivery_date) .`)

⚡ **Q73. “The export needs dates formatted as DD-Mon-YYYY. How do you do that?”**

```
SELECT TO_CHAR(order_date, 'DD-Mon-YYYY') AS formatted_date FROM
orders;
```

- **What it does:** Converts a raw date into a specific text format.
- **Why:** Reports/exports usually need a display format matching business conventions.
- **How it works:** `TO_CHAR` takes a format mask template and produces a matching string. (SQL Server: `FORMAT()` ; MySQL: `DATE_FORMAT()` .)

⚡ **Q74. “Some customer names have extra spaces around them from manual entry — how would you clean that up?”**

```
SELECT TRIM(name) AS clean_name FROM customers;
```

- **What it does:** Strips leading/trailing whitespace, leaving internal spaces untouched.
- **Why:** Accidental spaces make `"Ravi "` different from `"Ravi"` to a computer, silently breaking joins/grouping.
- **How it works:** `TRIM()` scans from both ends inward, stripping spaces until it hits real content.

⚡ **Q75. “This raw import uses dashes in dates but we need slashes — how would you fix that?”**

```
SELECT REPLACE(date_string, '-', '/') AS fixed_date FROM
raw_import;
```

- **What it does:** Swaps every dash for a slash in a text value.
- **Why:** Different systems use different date separators — standardizing is needed before converting text to a real date type.
- **How it works:** `REPLACE(column, old, new)` swaps EVERY occurrence, not just the first.

⚡ **Q76. “Can you pull just the domain out of each customer’s email address?”**

```
SELECT email, SUBSTRING(email FROM POSITION('@' IN email) + 1) AS
domain
FROM customers;
```

- **What it does:** Extracts everything after the “@” symbol.
- **Why:** Useful for analyzing which email providers customers use.
- **How it works:** `POSITION('@' IN email)` finds the index of “@”; `+1` moves past it; `SUBSTRING(... FROM that_position)` extracts the rest.

⚡ **Q77. “The external system needs order IDs as fixed 6-digit codes with leading zeros. How would you generate that?”**

```
SELECT LPAD(order_id::TEXT, 6, '0') AS padded_order_id FROM
orders;
```

- **What it does:** Converts a number into a fixed-width text string, padding with zeros.
- **Why:** Some external systems require IDs of a specific fixed length.
- **How it works:** `order_id::TEXT` converts to text first; `LPAD(string, length, char)` pads to the left until it reaches the target length.

🔗 **Q78. “I suspect some names in this table are actually corrupted junk data — how would you catch abnormally long ones?”**

```
SELECT name, LENGTH(name) AS name_length
FROM customers WHERE LENGTH(name) > 50;
```

- **What it does:** Flags unusually long name values.
- **Why:** A basic data-quality check catching obvious data-entry errors.
- **How it works:** `LENGTH()` counts total characters in the string.

🔗 **Q79. “How do you log the exact time a pipeline step completed?”**

```
INSERT INTO audit_log (event, event_ts) VALUES ('Load Complete',
CURRENT_TIMESTAMP);
```

- **What it does:** Records the exact moment the statement runs.
- **Why:** Every pipeline needs an audit trail for debugging and monitoring.
- **How it works:** `CURRENT_TIMESTAMP` (or `NOW()`) is filled in automatically by the database at execution time.

🔗 **Q80. “The finance team is seeing ugly numbers like 499.9999997 — how would you clean that up?”**

```
SELECT order_id, ROUND(amount, 2) AS amount_rounded FROM orders;
```

- **What it does:** Rounds to exactly 2 decimal places.
- **Why:** Floating-point math from multiplication/division can produce messy currency values.
- **How it works:** `ROUND(number, decimals)` rounds to the specified number of digits after the decimal.

📁 Round 9: Data Engineering Specific Scenarios

Interviewers weight this section heavily for actual Data Engineer roles — expect these constantly.

🔗 **Q81. “How would you handle a load where some customers are brand new and some already exist — without creating duplicates?”**

```
MERGE INTO target_customers t
USING staging_customers s
ON t.customer_id = s.customer_id
WHEN MATCHED THEN UPDATE SET t.name = s.name, t.city = s.city
WHEN NOT MATCHED THEN INSERT (customer_id, name, city) VALUES
(s.customer_id, s.name, s.city);
```

- **What it does:** Updates existing customers, inserts new ones, in one statement.
- **Why:** THE most important pattern in modern Data Engineering — idempotent, re-runnable, safe.
- **How it works:** `USING ... ON` finds matches; `WHEN MATCHED`

updates; `WHEN NOT MATCHED` inserts. Running it twice on the same data gives the same correct result both times — no duplicates.

🔗 Q82. “A customer’s city changed. We need to keep the OLD value for historical reporting, not just overwrite it. How?”

```
MERGE INTO dim_customer t
USING staging_customer s
ON t.customer_id = s.customer_id AND t.is_current = TRUE
WHEN MATCHED AND (t.city <> s.city) THEN
    UPDATE SET t.is_current = FALSE, t.end_date = CURRENT_DATE
WHEN NOT MATCHED THEN
    INSERT (customer_id, city, start_date, end_date, is_current)
    VALUES (s.customer_id, s.city, CURRENT_DATE, NULL, TRUE);
```

- **What it does:** Closes out the old row (marks not-current) instead of overwriting it; a new row (inserted separately) holds the updated value.
- **Why:** This is SCD Type 2 — a heavily tested Fabric/DP-700 concept for preserving full history.
- **How it works:** When a change is detected, the OLD row is flagged `is_current = FALSE` with an `end_date` — its data is never touched. A second INSERT pass then adds the new current version with `start_date = today`.

🔗 Q83. “Bronze layer has multiple versions of the same customer. How do you get a clean Silver table with just the latest?”

```
SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY
updated_at DESC) AS rn
    FROM bronze_customers
) t WHERE rn = 1;
```

- **What it does:** Same `ROW_NUMBER()` dedup as Q56, applied to a Bronze→Silver transformation.
- **Why:** Bronze is raw and can have multiple versions; Silver should be clean, one row per entity.
- **How it works:** Partition by the natural key, order by most recent, keep rank 1 — this becomes the Silver table.

🔗 Q84. “Before this data hits the Gold layer, how do you make sure nothing critical is missing?”

```
SELECT * FROM silver_orders
WHERE customer_id IS NULL OR order_date IS NULL OR amount IS NULL;
```

- **What it does:** Finds rows still missing required fields.
- **Why:** Gold layer must be fully trustworthy for reporting — never let broken rows silently flow into dashboards.
- **How it works:** Matching rows are typically routed to a quarantine table for review, while clean rows continue to Gold.

🔗 Q85. “Sometimes data shows up in our system days after the event actually happened. How would you detect that?”

```
SELECT * FROM staging_events
WHERE load_date - event_date > 7;
```

- **What it does:** Flags records with a large gap between when something happened and when it was loaded.

- **Why:** Late-arriving data (common in IoT, offline mobile apps) can break assumptions and needs special reprocessing.
- **How it works:** Simple date subtraction against a defined threshold (7 days here).

🔗 **Q86. “How does the pipeline know where to pick up from on the next run?”**

```
SELECT MAX(updated_at) AS new_watermark FROM staging_orders;
```

- **What it does:** Finds the most recent timestamp across everything just processed.
- **Why:** The other half of the incremental-loading pattern from Q70 — recording “how far we got.”
- **How it works:** This value gets written into a control table, becoming the “last_load_ts” the next run filters against.

🔗 **Q87. “The report needs category revenue laid out as separate columns per category, not stacked rows. How?”**

```
SELECT month,
       SUM(CASE WHEN category = 'Electronics' THEN revenue ELSE 0 END)
AS electronics,
       SUM(CASE WHEN category = 'Clothing' THEN revenue ELSE 0 END) AS
clothing
FROM monthly_category_revenue
GROUP BY month;
```

- **What it does:** Reshapes long-format data (row per month+category) into wide format (one row per month, category as columns).
- **Why:** Spreadsheet-style reports and many BI visuals expect categories as columns.
- **How it works:** `CASE WHEN` isolates one category’s revenue per column (contributing 0 for non-matching rows); `SUM` then collapses to one row per month with each category correctly separated.

🔗 **Q88. “This wide table has a column per month — can you convert it back to rows for easier analysis?”**

```
SELECT customer_id, 'Jan' AS month, jan_sales AS sales FROM
wide_table
UNION ALL
SELECT customer_id, 'Feb', feb_sales FROM wide_table;
```

- **What it does:** The reverse of Q87 — turns month columns into month rows.
- **Why:** BI tools and analytical queries strongly prefer long-format data over wide.
- **How it works:** Each `SELECT` grabs one wide column, relabels it generically, and adds a literal label identifying which month it represents; `UNION ALL` stacks them.

🔗 **Q89. “Someone added a new column upstream and it’s not in our target table yet. How would you detect that automatically?”**

```

SELECT column_name FROM information_schema.columns
WHERE table_name = 'staging_orders'
AND column_name NOT IN (
    SELECT column_name FROM information_schema.columns WHERE
table_name = 'target_orders'
);

```

- **What it does:** Compares table structures, finds columns present in staging but missing in target.
- **Why:** “Schema drift” detection catches upstream changes before they break the pipeline or silently drop data.
- **How it works:** `information_schema.columns` is a system view holding metadata about tables; this query applies familiar `NOT IN` logic to column NAMES instead of regular data.

🔗 Q90. “How do you confirm yesterday’s load actually landed the expected number of rows?”

```

SELECT partition_date, COUNT(*) AS row_count
FROM gold_sales
WHERE partition_date = CURRENT_DATE - 1
GROUP BY partition_date;

```

- **What it does:** Counts rows for yesterday’s specific partition.
- **Why:** An automated sanity check — a sudden drop or spike signals a broken upstream feed or duplicate load.
- **How it works:** Filtering by `partition_date` on a partitioned table only scans that day’s data, making both the load and this check fast; the count is usually compared against a historical baseline to detect anomalies.

📁 Round 10: Real-World Business Scenarios & Performance

🔗 Q91. “Find customers who bought Product A but never bought Product B — for a targeted campaign.”

```

SELECT DISTINCT o.customer_id
FROM orders o JOIN order_items oi ON o.order_id = oi.order_id
WHERE oi.product_id = 'A'
AND o.customer_id NOT IN (
    SELECT o2.customer_id FROM orders o2 JOIN order_items oi2 ON
o2.order_id = oi2.order_id
WHERE oi2.product_id = 'B'
);

```

- **What it does:** Finds A-buyers who have never bought B.
- **Why:** Classic cross-sell targeting — customers likely to want B based on their interest in A.
- **How it works:** Main query finds A-buyers; subquery separately finds B-buyers; `NOT IN` excludes them. ⚠ Same NULL trap as Q43 applies — production code often uses `NOT EXISTS` instead for safety.

🔗 Q92. “What are the top 3 best-selling products in EACH category?”

```

SELECT * FROM (
    SELECT category, product_name, SUM(quantity) AS total_qty,
           RANK() OVER (PARTITION BY category ORDER BY SUM(quantity)
DESC) AS rnk
    FROM order_items oi JOIN products p ON oi.product_id =
p.product_id
    GROUP BY category, product_name
) t WHERE rnk <= 3;

```

- **What it does:** Finds the top 3 products by quantity, per category.
- **Why:** One of the most frequently asked SQL interview questions — tests GROUP BY + window function ranking together.
- **How it works:** `PARTITION BY category` resets ranking per category; `rnk <= 3` keeps only the podium finishers in each.

🔗 Q93. “How many of the customers who joined in a given month were still buying 1, 2, 3 months later?”

```

WITH first_purchase AS (
    SELECT customer_id, MIN(DATE_TRUNC('month', order_date)) AS
cohort_month
    FROM orders GROUP BY customer_id
)
SELECT fp.cohort_month, DATE_TRUNC('month', o.order_date) AS
activity_month,
       COUNT(DISTINCT o.customer_id) AS active_customers
FROM orders o JOIN first_purchase fp ON o.customer_id =
fp.customer_id
GROUP BY fp.cohort_month, DATE_TRUNC('month', o.order_date);

```

- **What it does:** Groups customers by their first-purchase month (cohort), tracks activity in later months.
- **Why:** Cohort analysis is a standard technique for understanding retention/loyalty over time.
- **How it works:** The CTE finds each customer’s cohort; the main query counts distinct active customers per (cohort, activity month) combination.

🔗 Q94. “This query has gotten really slow — how would you start investigating why?”

```

EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_id = 101;

```

- **What it does:** Shows the actual execution plan and real timing/row counts.
- **Why:** You can’t fix performance by guessing — you need to see whether the engine is scanning efficiently or wastefully.
- **How it works:** `EXPLAIN` shows the planned strategy; `EXPLAIN ANALYZE` actually runs it and reports real numbers — e.g., “Seq Scan” (full table scan, slow) vs. “Index Scan” (fast, targeted).

🔗 Q95. “This filter on customer_id is scanning the whole table every time. What would you do?”

```

CREATE INDEX idx_orders_customer_id ON orders(customer_id);

```

- **What it does:** Builds a sorted lookup structure on `customer_id`.
- **Why:** Without an index, filtering means checking every row one by one; with an index, it’s like jumping straight to the right page in a sorted phone book.
- **How it works:** Any column frequently used in WHERE/JOIN/GROUP

BY is a good indexing candidate — though indexes add storage cost and slightly slow down writes, so you don't index everything blindly.

🔗 Q96. “I wrote `WHERE YEAR(order_date) = 2026` and it's slow even with an index on `order_date`. Why?”

```
-- ✗ Slow (function on column disables index use):
SELECT * FROM orders WHERE YEAR(order_date) = 2026;

-- ✔ Fast (sargable — index-friendly):
SELECT * FROM orders WHERE order_date >= '2026-01-01' AND
order_date < '2027-01-01';
```

- **What it does:** Both return the same rows, but perform very differently.
- **Why:** Wrapping a column in a function forces the engine to calculate that function for EVERY row before filtering, completely bypassing the index.
- **How it works:** A condition is “sargable” when the raw column stays untouched on one side of the comparison, letting the index actually be used. This exact mistake is one of the most common real production performance bugs.

🔗 Q97. “How would you confirm whether this orders table actually has duplicates, before I trust any dedup fix?”

```
SELECT customer_id, order_date, COUNT(*) AS dup_count
FROM orders
GROUP BY customer_id, order_date
HAVING COUNT(*) > 1;
```

- **What it does:** Finds combinations of `customer_id` + `order_date` appearing more than once.
- **Why:** A diagnostic step, run BEFORE applying a dedup fix like Q56's pattern.
- **How it works:** Grouping by the combination (not each column separately) treats matching pairs as one group; `HAVING COUNT(*) > 1` isolates true duplicates.

🔗 Q98. “Show me year-to-date revenue as of each order, resetting every January.”

```
SELECT order_date, amount,
SUM(amount) OVER (
    PARTITION BY EXTRACT(YEAR FROM order_date)
    ORDER BY order_date
) AS ytd_revenue
FROM orders;
```

- **What it does:** A running total that resets at the start of each calendar year.
- **Why:** YTD is one of the most requested finance metrics.
- **How it works:** Combines partitioning (reset per year, like Q51) with an ordered running sum (like Q53) in one window function.

🔗 Q99. “Which customers were active in BOTH Q1 and Q2?”

```
SELECT customer_id FROM orders WHERE order_date BETWEEN '2026-01-01' AND '2026-03-31'
INTERSECT
SELECT customer_id FROM orders WHERE order_date BETWEEN '2026-04-01' AND '2026-06-30';
```

- **What it does:** Returns customer_ids present in both quarterly result sets.
- **Why:** Cleanly expresses “AND present in both” without a more complex join.
- **How it works:** `INTERSECT` runs both SELECTs independently and returns only rows appearing in both.

🔗 **Q100. “We have online orders and in-store orders in separate tables — combine them into one unified, duplicate-free report.”**

```
SELECT order_id, customer_id, amount, 'Online' AS source FROM
online_orders
UNION
SELECT order_id, customer_id, amount, 'In-Store' AS source FROM
store_orders;
```

- **What it does:** Stacks two tables into one result, removing exact duplicate rows.
- **Why:** Companies store channel data separately, but unified reporting needs it combined.
- **How it works:** Both SELECTs need matching column counts/types; `UNION` additionally checks and removes duplicates (extra work compared to `UNION ALL`, which just stacks without checking — use `UNION ALL` when you’re confident no genuine duplicates can occur, for better performance).

📖 How to Practice This Effectively

1. Read only the 🔗 question first — try answering OUT LOUD as if in a real interview, before looking at the query.
 2. Practice explaining the “Why” and “How” in your own words — interviewers care more about your reasoning than the exact syntax.
 3. Focus extra time on **Round 6 (Window Functions)** and **Round 9 (Data Engineering Scenarios)** — these get asked the most for real DE roles and DP-700.
 4. Pay close attention to every “⚠️ Gotcha” — interviewers love probing exactly these edge cases to separate real understanding from memorization.
 5. Re-attempt anything you got wrong after 2-3 days.
-